Priority Queue : Different implementations

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations

1. **Implementation of priority queue: Using an Ordered List**

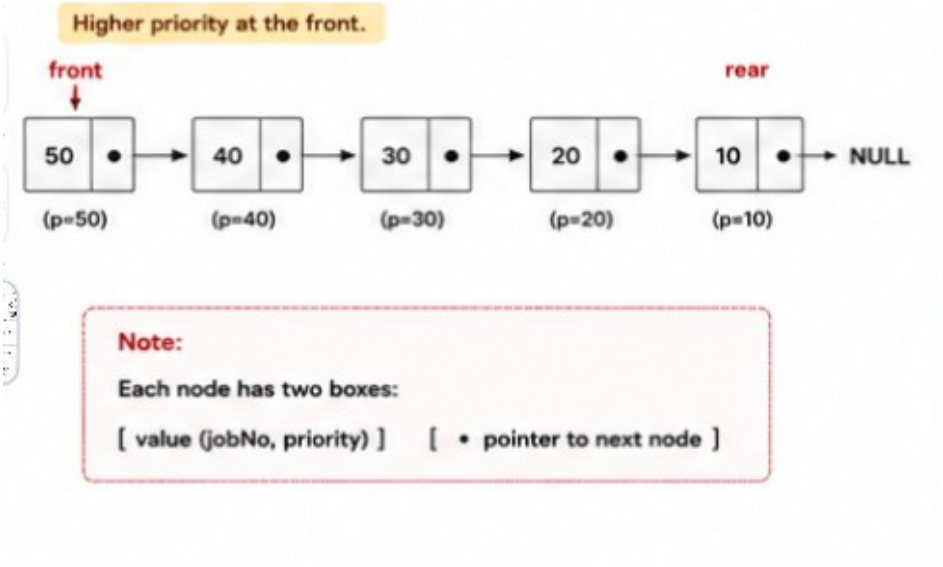
We have discussed the implementation of priority queue using an unordered list. Now, we shall discuss the priority queue using an ordered list. The queue structure will remain the same. The node structure will remain the same. The interface will also remain the same!

```
struct queue
{
        node_t* front_;
        node_t* rear_;
};
typedef struct queue queue_t;

struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;

void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.



Initialize:

No change. Both the front and the rear of the queue are grounded.

```
 void init_queue(queue_t *ptr_queue)
{
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Deinitialize :

No change. This is one of the ways of removing all the elements in the queue. Deque each element until the queue becomes empty.

The following questions are still valid.

As deque is a costly operation, we could do it better. Can you try it out?

Look at this code below. Do you think this is required?

```
ptr_queue->rear_ = ptr_queue->front_ = NULL;
void deinit_queue(queue_t *ptr_queue)
{
        while(! is_empty(ptr_queue))
        {
                deque(ptr_queue);
        }
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Is full :

No change. This is a trick. Try allocating. If it succeeds, then there is memory available. Release the memory allocated just for the testing purpose.

```
int is_full(queue_t *ptr_queue)
{
   node_t* temp;
  temp = (node_t*)malloc(sizeof(node_t));
   if (temp == NULL) {
      return 1;
   }
   free(temp);
   return 0;
}
```

Is empty:

No change. Check whether the front of the queue is NULL.

Can we check the rear of the queue in stead?

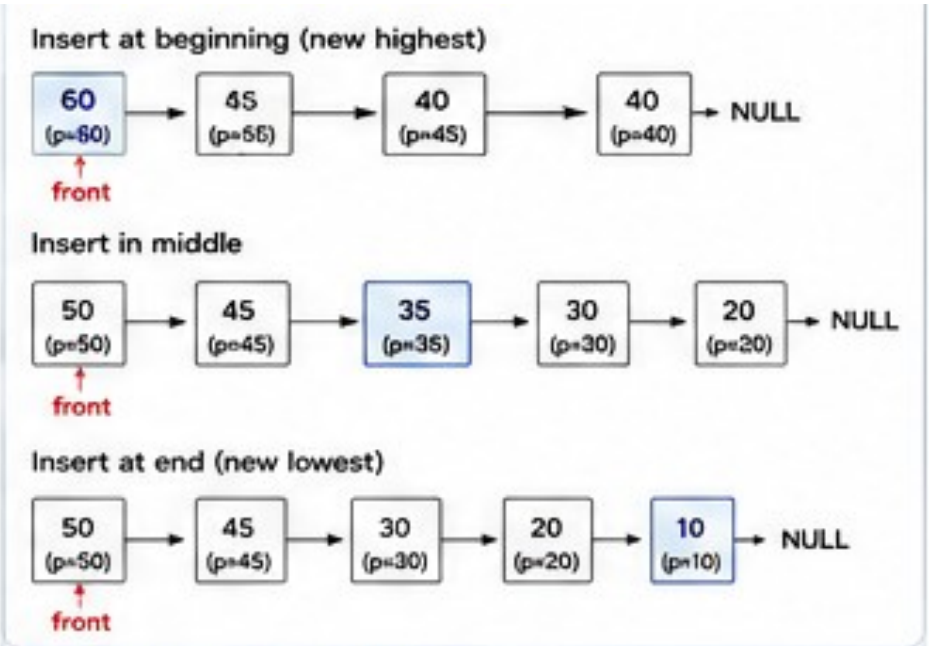
Would front and rear being equal indicate that the queue is empty?

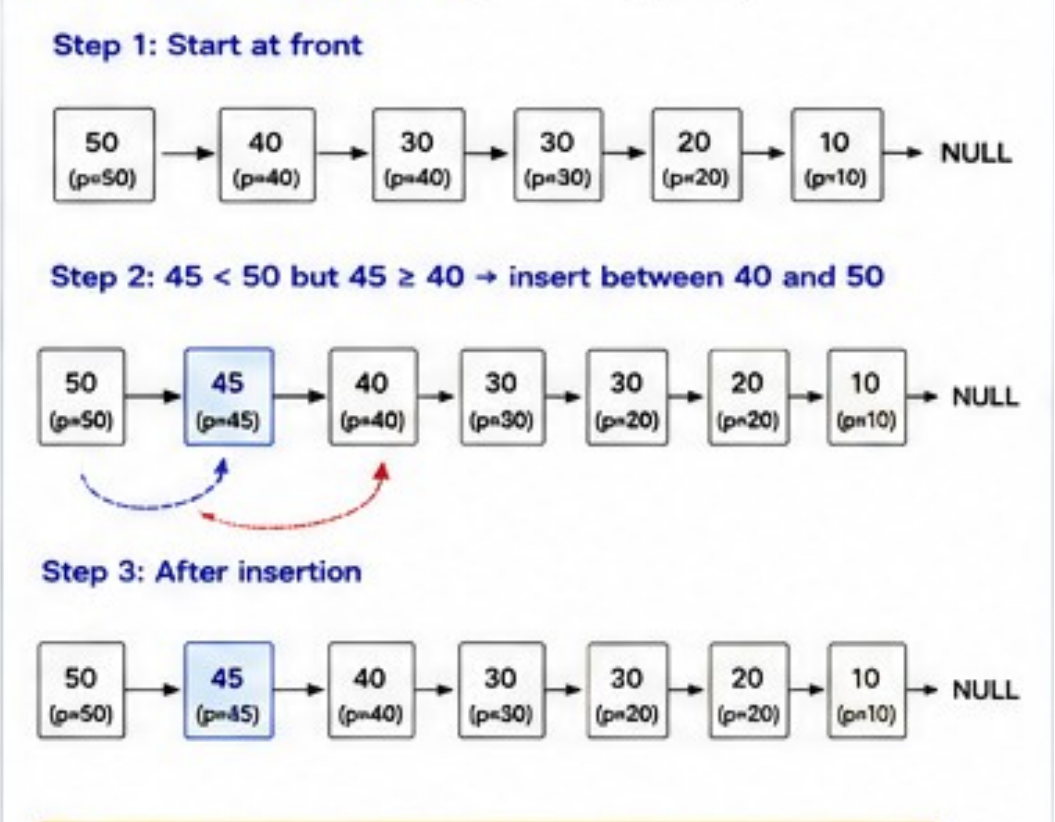
```
int is_empty(queue_t *ptr_queue)
{
        return ptr_queue->front_ == NULL;
}
```

Enque :

This operation does consider the priority while inserting the node.

If queue is full, the program is exited.





We will see to it that the node with the highest priority is always at the front.

What if more than one node has the same high priority. We will insert before the one with the same priority so that the newly added job gets services last amongst those with the same priority.

Make a node. Populate the node. Find the position to insert.

If the queue is empty, add at the rear. Also change the front to point to the new node created.

Otherwise traverse using two pointers prev and pres.

Compare the priority of the present node with that of the new node to decide whether we have found the position to insert.

Observe the relational operator >=. Would this work if the operator is changed to > ?

while(pres != NULL && get_priority(&pres->jobx_) >= get_priority(&temp->jobx_))

Once the location is found, check whether the insertion is in the beginning or the end or anywhere in between.

Make the changes in the queue as well as the new node added.

```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{
	if(! is_full(ptr_queue))
	{
		node_t* temp = create_node(ptr_job);
		// empty queue
		if(ptr_queue->rear_ == NULL)
		{
			ptr_queue->front_ = temp;
			ptr_queue->rear_ = temp;
			temp->next_ = NULL;
		}
		else // find the location to insert
		{
			node_t* prev = NULL; node_t* pres = ptr_queue->front_;
			while(pres != NULL &&
				get_priority(&pres->jobx_) >= get_priority(&temp->jobx_))
			{
				prev = pres;
				pres = pres->next_;
			}
			if(prev == NULL)
			{
				ptr_queue->front_ = temp;
			}
			else
```

```
                {
                        prev->next_ = temp;
                        if(pres == NULL)
                        {
                                ptr_queue->rear_ = temp;
                        }
                }
                temp->next_ = pres;
        }
    }
    else
    {
        printf("queue full; cannot push\n");
        exit(1);
    }
}
```

Deque :

As the priority queue is in an ordered list, the deque is straight forward. Remove the node to which front points to.

If the queue is empty, exit the program.

Otherwise, copy the pointer to the first node to a local variable.

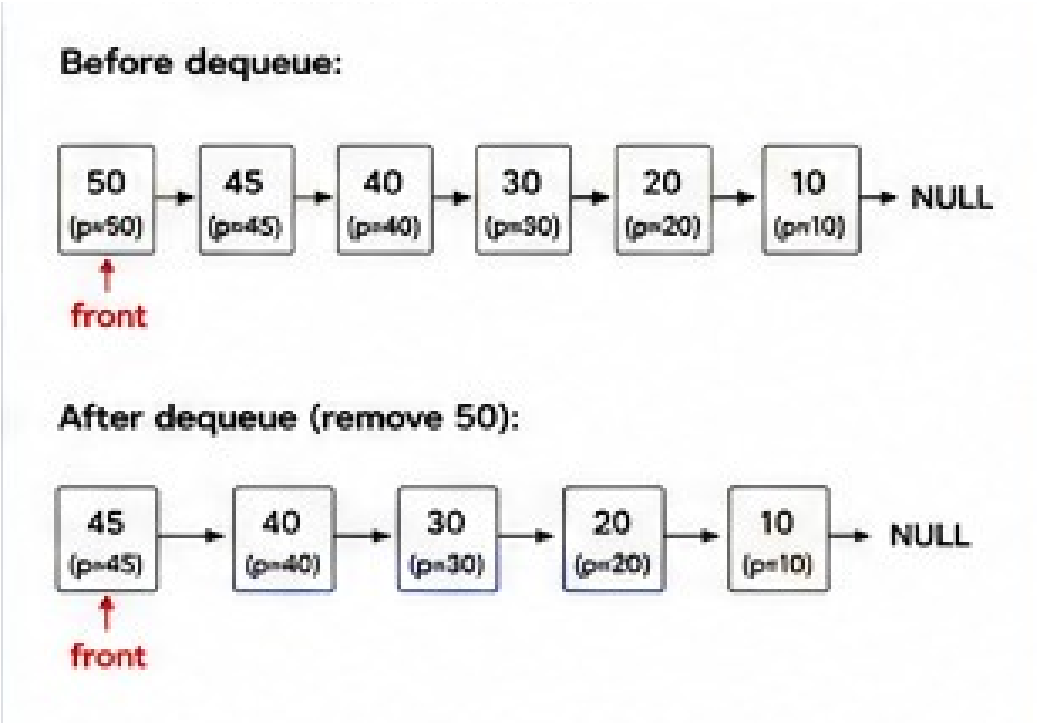
Copy the job to a local variable.

Change front to point to the node after the first.

If front becomes NULL, change the rear too.

Release the memory of the first node whose pointer has been copied.

Returned the job held in a local variable.




```
job_t deque(queue_t *ptr_queue)

{

        if(! is_empty(ptr_queue))

        {


                job_t jobx;  node_t* temp = ptr_queue->front_;

                jobx = temp->jobx_;


                ptr_queue->front_ = temp->next_;

                if(ptr_queue->front_ == NULL)

                {

                        ptr_queue->rear_ = NULL;

                }

                free(temp);
```

```
        return jobx;
    }
    else
    {
        printf("queue empty; cannot pop\n");
        exit(1);
    }
}
```

Efficiency of operations :

Enque : This requires walking through the list to find the node with the highest priority and then insert. So, the number of operations in the worst case is proportional to the number of nodes in the queue

Deque :. This requires a few operations. We remove from the front without considering the priority.